

Travaux Pratiques GitLab CI/CD

Introduction à l'intégration continue

ISTA Mirleft

Objectifs

Ce TP vous permettra de découvrir GitLab et son système d'intégration continue (CI/CD). Vous allez apprendre à :

- Comprendre les différences entre GitLab et GitHub
- Créer et gérer un projet sur GitLab
- Créer un pipeline CI/CD simple avec GitLab CI
- Exécuter et surveiller un pipeline
- Comprendre le rôle des runners GitLab

Prérequis

- Connaissances de base en Git et GitHub
 - Un compte GitLab créé (gratuit sur <https://gitlab.com>)
 - Git installé sur votre machine
 - Un éditeur de texte (VS Code)
-

1 Partie 1 : GitLab vs GitHub

1.1 Comparaison

Question 1.1 - Recherchez et listez 3 différences principales entre GitLab et GitHub.

Question 1.2 - Qu'est-ce que le CI/CD ? Pourquoi est-il important dans le développement logiciel ?

Question 1.3 - GitLab offre un système CI/CD intégré. Comment s'appelle le fichier de configuration pour GitLab CI ?

2 Partie 2 : Création de projet sur GitLab

Dans cette partie, vous allez découvrir 3 méthodes pour créer un projet sur GitLab.

2.1 Méthode 1 : Initialisation locale puis liaison à GitLab

Question 2.1 - Sur votre machine, créez un répertoire nommé `calculatrice-web` et initialisez-y un dépôt Git.

Question 2.2 - Sur GitLab (<https://gitlab.com>), créez un nouveau projet vide nommé `calculatrice-web`. Ne cochez pas l'option "Initialize repository with a README".

Question 2.3 - Ajoutez le dépôt GitLab comme remote `origin` à votre dépôt local. GitLab vous fournit l'URL après création du projet.

2.2 Méthode 2 : Création sur GitLab puis clonage (Alternative)

Note : Si vous avez déjà créé le projet avec la Méthode 1, passez à la section suivante. Sinon :

Question 2.4 - Créez un nouveau projet sur GitLab avec l'option "Initialize repository with a README" cochée.

Question 2.5 - Clonez ce projet sur votre machine locale.

2.3 Méthode 3 : Import depuis GitHub (Information)

GitLab permet d'importer directement un projet depuis GitHub via l'option "Import project" > "GitHub". Cette fonctionnalité facilite la migration de projets.

3 Partie 3 : Création du projet web

Vous allez créer une calculatrice web simple avec HTML, CSS et JavaScript.

3.1 Structure du projet

Question 3.1 - Dans votre répertoire `calculatrice-web`, créez la structure suivante :

```
calculatrice-web/  
  index.html  
  style.css  
  script.js  
  test.js
```

3.2 Contenu des fichiers

Question 3.2 - Créez le fichier `index.html` avec le contenu suivant : (voir les fichiers envoyer)

Question 3.3 - Créez le fichier `style.css` avec le contenu suivant : (voir les fichiers envoyer)

Question 3.4 - Créez le fichier `script.js` avec le contenu suivant : (voir les fichiers envoyer)

Question 3.5 - Créez le fichier `test.js` avec le contenu suivant : (voir les fichiers envoyer)

Question 3.6 - Ajoutez tous les fichiers à Git, créez un commit avec le message "Ajout projet calculatrice", puis poussez vers GitLab.

4 Partie 4 : GitLab CI - Création du Pipeline

4.1 Comprendre le fichier .gitlab-ci.yml

Le fichier `.gitlab-ci.yml` définit votre pipeline CI/CD. Il contient :

- **stages** : Les étapes du pipeline (build, test, deploy...)
- **jobs** : Les tâches à exécuter dans chaque stage
- **script** : Les commandes à exécuter
- **image** : L'image Docker à utiliser (environnement)

4.2 Création du pipeline

Question 4.1 - À la racine de votre projet, créez un fichier nommé `.gitlab-ci.yml`

Question 4.2 - Dans ce fichier, définissez 3 stages dans cet ordre :

- **validate** : pour valider la structure du projet
- **test** : pour exécuter les tests
- **build** : pour préparer le projet

*Indice : Utilisez le mot-clé **stages**: suivi d'une liste*

Question 4.3 - Créez un job nommé **check-files** dans le stage **validate**. Ce job doit :

- Utiliser l'image : **alpine:latest**
- Exécuter ces commandes :
 - `ls -la`
 - `echo "Vérification de la structure du projet"`
 - `test -f index.html && echo " index.html trouvé"`
 - `test -f script.js && echo " script.js trouvé"`
 - `test -f test.js && echo " test.js trouvé"`

Structure d'un job :

```
nom_du_job:
  stage: nom_stage
  image: nom_image
  script:
    - commande1
    - commande2
```

Question 4.4 - Créez deux jobs dans le stage **test** :

Job 1 : run-tests

- Image : **node:16-alpine**
- Script :
 - `echo "Exécution des tests"`
 - `node test.js`

Job 2 : lint-check

- Image : **alpine:latest**
- Script :
 - `echo "Vérification du code"`

```
— wc -l *.js *.html *.css
— echo "Vérification terminée"
```

Question 4.5 - Créez un job nommé `package` dans le stage `build`. Ce job doit :

- Utiliser l'image : `alpine:latest`
- Exécuter ces commandes :
 - `echo "Préparation du build"`
 - `mkdir -p build`
 - `cp *.html *.css *.js build/`
 - `ls -la build/`
 - `echo "Build terminé avec succès"`

Question 4.6 - Vérifiez votre fichier `.gitlab-ci.yml`. Ajoutez-le à Git, commitez avec le message "Ajout pipeline CI", puis poussez vers GitLab.

5 Partie 5 : Exécution et Surveillance du Pipeline

5.1 Observation du pipeline

Question 5.1 - Sur GitLab, allez dans votre projet puis cliquez sur "CI/CD" > "Pipelines". Que voyez-vous ?

Question 5.2 - Cliquez sur le pipeline en cours d'exécution. Combien de stages sont affichés ? Combien de jobs par stage ?

Question 5.3 - Cliquez sur le job `check-files`. Observez les logs d'exécution. Quel est le statut du job ?

Question 5.4 - Attendez que tous les jobs se terminent. Le pipeline est-il "passed" (réussi) ou "failed" (échoué) ?

Question 5.5 - Dans quel ordre les stages se sont-ils exécutés ?

Question 5.6 - Les jobs d'un même stage s'exécutent-ils en séquence ou en parallèle ?

5.2 Modification et réexécution

Question 5.7 - Modifiez le fichier `index.html` (ajoutez un commentaire). Commitez et poussez. Un nouveau pipeline se lance-t-il automatiquement ?

Question 5.8 - Sur GitLab, vous pouvez relancer un pipeline manuellement. Trouvez le bouton "Run pipeline" et lancez un nouveau pipeline.

6 Partie 6 : Les Runners GitLab

6.1 Comprendre les runners

Question 6.1 - Qu'est-ce qu'un runner GitLab ? Recherchez une définition simple.

Question 6.2 - Sur GitLab, allez dans "Settings" > "CI/CD" > "Runners". Quel type de runner votre projet utilise-t-il actuellement ?

Question 6.3 - GitLab propose deux types de runners :

- **Shared runners** : Fournis gratuitement par GitLab
- **Specific runners** : Installés et gérés par vous

Quel est l'avantage d'un shared runner pour un petit projet ?

Question 6.4 - Dans les logs d'un job, trouvez la section qui indique quel runner a exécuté le job. Notez son nom ou ID.

7 Partie 7 : Questions de Synthèse

Question 7.1 - Expliquez avec vos mots ce qu'est un pipeline CI/CD et pourquoi il est utile.

Question 7.2 - Quels sont les 3 stages que vous avez créés dans votre pipeline ? Quel est le rôle de chacun ?

Question 7.3 - Que se passe-t-il si un test échoue dans le stage `test` ? Le stage `build` s'exécute-t-il quand même ?

Question 7.4 - Pourquoi utilise-t-on différentes images Docker (alpine, node) dans les jobs ?